

element Z in standard basis is written as

$$Z = \sum_{k=0}^7 z_k \alpha^k.$$

In dual basis, it is represented as

$$Z = \sum_{k=0}^7 z'_k \lambda_k$$

where

$$\begin{aligned} z'_k &= \text{Tr}(Z\alpha^k) \\ &= \text{Tr}((z_0\alpha^0 + z_1\alpha^1 + z_2\alpha^2 + z_3\alpha^3 + z_4\alpha^4 \\ &\quad + z_5\alpha^5 + z_6\alpha^6 + z_7\alpha^7)\alpha^k) \\ &= z_0 \text{Tr}(\alpha^k) + z_1 \text{Tr}(\alpha^{k+1}) + z_2 \text{Tr}(\alpha^{k+2}) + z_3 \text{Tr}(\alpha^{k+3}) \\ &\quad + z_4 \text{Tr}(\alpha^{k+4}) + z_5 \text{Tr}(\alpha^{k+5}) + z_6 \text{Tr}(\alpha^{k+6}) + z_7 \text{Tr}(\alpha^{k+7}). \end{aligned}$$

Therefore, once $\text{Tr}(\alpha^k)$, for $0 \leq k \leq 14$, are known, the basis conversion from standard to dual can be completed.

APPENDIX B

This Appendix describes a method for converting an element represented in dual basis to standard basis. Again, let an element Z in dual basis be written as

$$Z = \sum_{k=0}^7 z'_k \cdot \lambda_k.$$

In standard basis, let it be represented as

$$Z = \sum_{k=0}^7 z_k \cdot \alpha^k.$$

From the definition of the trace, one obtains

$$\begin{aligned} z_k &= \text{Tr}(Z\lambda_k) \\ &= \text{Tr}((z'_0\lambda_0 + z'_1\lambda_1 + z'_2\lambda_2 + z'_3\lambda_3 + z'_4\lambda_4 \\ &\quad + z'_5\lambda_5 + z'_6\lambda_6 + z'_7\lambda_7)\lambda_k) \\ &= z'_0 \text{Tr}(\lambda_0\lambda_k) + z'_1 \text{Tr}(\lambda_1\lambda_k) + z'_2 \text{Tr}(\lambda_2\lambda_k) + z'_3 \text{Tr}(\lambda_3\lambda_k) \\ &\quad + z'_4 \text{Tr}(\lambda_4\lambda_k) + z'_5 \text{Tr}(\lambda_5\lambda_k) + z'_6 \text{Tr}(\lambda_6\lambda_k) + z'_7 \text{Tr}(\lambda_7\lambda_k). \end{aligned}$$

Hence, if the dual basis is determined and trace values of the above are calculated, the basis conversion from dual basis to standard basis can be completed.

APPENDIX C

In this Appendix, a fast algorithm for calculating the trace values of elements in finite field $GF(2^m)$ is described. From the definition of the trace one has, for $\beta, \beta^2 \in GF(2^m)$,

$$\begin{aligned} \text{Tr}(\beta^2) &= \sum_{k=0}^{m-1} (\beta^2)^{2^k} = \beta^2 + \beta^{2^2} + \cdots + \beta^{2^m} \\ &= \beta + \beta^2 + \cdots + \beta^{2^{m-1}} = \text{Tr}(\beta). \end{aligned}$$

Hence, if $\text{Tr}(\beta)$ is obtained, then $\text{Tr}(\beta^2)$ can also be obtained without calculation.

Since every element in $GF(2^m)$ can be represented by the elements which compose the basis, i.e., for $GF(2^m)$, and the basis is $\{\alpha^0, \alpha^1,$

$\alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6, \alpha^7\}$, then β can be written as

$$\beta = \beta_0\alpha^0 + \beta_1\alpha^1 + \beta_2\alpha^2 + \beta_3\alpha^3 + \beta_4\alpha^4 + \beta_5\alpha^5 + \beta_6\alpha^6 + \beta_7\alpha^7.$$

From the properties of the trace, one has

$$\begin{aligned} \text{Tr}(\beta) &= \beta_0 \text{Tr}(\alpha^0) + \beta_1 \text{Tr}(\alpha^1) + \beta_2 \text{Tr}(\alpha^2) + \beta_3 \text{Tr}(\alpha^3) \\ &\quad + \beta_4 \text{Tr}(\alpha^4) + \beta_5 \text{Tr}(\alpha^5) + \beta_6 \text{Tr}(\alpha^6) + \beta_7 \text{Tr}(\alpha^7). \end{aligned}$$

Hence it is only necessary to calculate trace values of $\alpha^0, \alpha^1, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6, \alpha^7$; the rest can be obtained easily once it is represented by the basis elements.

REFERENCES

- [1] E. R. Berlekamp, "Bit-serial Reed-Solomon encoders," *IEEE Trans. Inform. Theory*, vol. IT-28, pp. 869-874, Nov. 1982.
- [2] C. C. Wang, T. K. Truong, H. M. Shao, L. J. Deutsch, J. K. Omura, and I. S. Reed, "VLSI architecture for computing multiplications and inverses in $GF(2^m)$," *IEEE Trans. Comput.*, vol. C-34, Aug. 1985.
- [3] P. A. Scott, S. E. Tarvares, and L. E. Peppard, "A fast multiplier for $GF(2^m)$," *IEEE J. Select. Areas Commun.*, vol. SAC-4, Jan. 1986.
- [4] I. S. Hsu, I. S. Reed, T. K. Truong, K. Wang, C. S. Yeh, and L. J. Deutsch, "The VLSI implementation of a Reed-Solomon encoder using Berlekamp's bit-serial multiplier algorithm," *IEEE Trans. Comput.*, vol. C-33, Oct. 1984.
- [5] H. M. Shao, T. K. Truong, L. J. Deutsch, J. H. Yuen, and I. S. Reed, "A VLSI design of a pipeline Reed-Solomon decoder," *IEEE Trans. Comput.*, vol. C-34, May 1985.
- [6] M. Perlman and J. J. Lee, "A comparison of conventional Reed-Solomon encoders and Berlekamp's architecture," NASA Tech. Brief 3610-81-119, Jet Propulsion Lab., Pasadena, CA, July 10, 1981.
- [7] C. C. Wang, "Computer simulation of finite field multiplications based on Massey-Omura's normal basis representation of field elements," private communication, 1985.

Approximate Analysis of Fork/Join Synchronization in Parallel Queues

R. NELSON AND A. N. TANTAWI

Abstract—A new approximation technique, called the *scaling approximation*, is introduced and applied to the analysis of homogeneous fork/join queueing systems consisting of $K \geq 2$ servers. The development of the scaling approximation technique is guided by both experimental and theoretical considerations. The essence of the approximation stems from an interesting observation that there exist upper and lower bounds on the mean response time which grow at the same rate as a function of K . Simple, closed-form approximate expressions for the mean response time are derived and compared to simulation results. The relative error in the approximation is less than 5 percent for $K \leq 32$.

Index Terms—Fork/join queues, modeling and analysis, parallel queues, scaling approximation, synchronization.

I. INTRODUCTION

Technological limitations on the speed of uniprocessor computer systems have led to the emergence of systems with a multitude of

Manuscript received November 12, 1985; revised August 26, 1986.

The authors are with the IBM Thomas J. Watson Research Center, Yorktown Height, NY 10598.

IEEE Log Number 8716250.

processing units. In such systems, a job is divided into a number of tasks, each executing on a processing unit concurrently with other tasks. The division of a job into a set of tasks is governed by the logical structure of the job and the interdependences among tasks. This is usually described by a computation graph where nodes in the graph represent tasks and directed edges represent precedence relations. Two main features of computation graphs are *fork nodes* where more than one edge emanates from the node and *join nodes* where more than one edge enters the node. An important performance measure of parallel processing systems is the execution time of a job described by a computation graph.

A simple computation graph consists of a single fork node, a single join node, and a number of single-threaded nodes forming parallel paths between the fork and join nodes. Heidelberger and Trivedi [5] consider a queueing network model of a computer system and a computation graph which consists of fork nodes but not join nodes, i.e., synchronization among tasks is not required. The analysis of such a model is analytically intractable due to the parallelism of tasks. They develop an approximate solution method along with bounds on performance. In [6], the model is extended to contain a join node (the same as the fork node) and two approximation techniques are presented: one is based on a decomposition approach and the other involves an iterative solution of a sequence of mathematically tractable queueing networks.

Exact analysis is possible for simpler system models and computation graphs. Flatto and Hahn [3] consider a graph with a single fork node, a single join node, and exactly two tasks, one in each path between the fork and join nodes. The system consists of two heterogeneous processing units (servers), each having its own queue. The execution time of a task is exponentially distributed. Assuming a Poisson arrival process of jobs, they obtain a double transform of the system occupancy and asymptotic formulas as either one of the queues becomes long. Flatto [4] derives limit laws for the expectation and distribution for either of the queue lengths conditioned on the other. A more general model is considered by Baccelli and Makowski [1] where the system consists of $K \geq 2$ heterogeneous servers and the arrival and service processes are general. They obtain bounds on the response time of a job in such a system. An upper bound assumes K mutually independent GI/G/1 parallel queueing systems whereas a lower bound assumes D/G/1 parallel queueing systems. The tightness of these bounds is not investigated.

In this paper, we consider a fork/join queueing system with $K \geq 2$ homogeneous exponential servers and obtain very simple approximate expressions for the mean job response time. Our approximation approach is based on a *scaling approximation* technique which we develop. The essence of the approximation stems from the observation that there exist upper and lower bounds which grow at the same rate as a function of K . By comparison to simulation results, the relative error in our approximation is less than 5 percent for $K \leq 32$.

This paper is organized as follows. In Section II, we describe the fork/join queueing model. The approximation technique is presented in Section III where we obtain an approximate closed-form expression for the mean job response time and compare it to simulation results. In Section IV, we apply our approximation technique to two areas: 1) the design of multiprocessor systems and 2) the performance of database systems. Further research problems and possible extensions are discussed in Section V.

II. MODEL DESCRIPTION

We consider a system which consists of K identical parallel queueing systems as shown in Fig. 1. Jobs arrive in a time-invariant, state-independent Poisson fashion with rate λ . Upon arrival, a job *forks* into K tasks. Task k , $k = 1, 2, \dots, K$, is assigned to the k th queueing system which consists of a single server and an infinite capacity queue. The service times of tasks are independent and exponentially distributed with mean $1/\mu$. The server utilization is denoted by $\rho = \lambda/\mu$. A job leaves the system as soon as all its tasks complete their service. In other words, tasks corresponding to the same job are *joined* before departing the system. We are interested in the mean job response time which we denote by T_K .

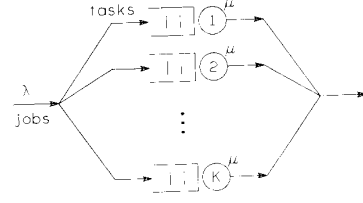


Fig. 1. Fork/join queueing model.

A variation of this model is considered in Section IV where an independent arrival process is added to each queueing system. Other extensions such as the feedback of jobs, general service times, and heterogeneous servers are subjects of further research.

III. APPROXIMATE ANALYSIS

The state of the system described in Section II may be represented by the number of tasks in each of the K queueing systems. This results in a K -dimensional Markov process whose analysis seems to be hard, if not impossible, for the general case. A special case, $K = 2$ and heterogeneous servers, has been considered in [3] and [4]. In Appendix B we derive the mean job response time in the case of two homogeneous queues; it is given by

$$T_2 = \frac{12 - \rho}{8} T_1 \quad (1)$$

where $T_1 = (1/\mu)/(1 - \rho)$ is the mean response time in an M/M/1 queue.

Our approach to solving the $K > 2$ queue problem is an approximate one. The essence of our approximation method stems from a very interesting observation of bounds on T_K . An upper bound is obtained by assuming independent queueing systems and, since the response times in M/M/1 queues are exponentially distributed with mean T_1 , is given by

$$T_K \leq H_K T_1 \quad (2)$$

where H_K is the harmonic series. A proof of this upper bound using associated random variables is provided in Appendix A. A lower bound, on the other hand, is obtained by neglecting queueing effects. In this case, the response time is the maximum of K i.i.d. exponentially distributed service times each with mean $1/\mu$. Thus, we have

$$T_K \geq H_K \frac{1}{\mu} \quad (3)$$

The tightness of these bounds is not of concern here. Looking at (2) and (3), one sees that both bounds grow at the same rate H_K . In other words, for large K the bounds are of order $O(\ln K)$. From this observation, we conclude that T_K itself must be growing at the same rate. Consequently, knowing the value of T_2 , we may write

$$T_K \approx S_K(\rho) T_2, \quad K \geq 2, \quad (4)$$

where $S_K(\rho)$ is a scaling factor which grows at rate $O(\ln K)$. We seek an approximation of the following form: $S_K(\rho) = \alpha(\rho) + \beta(\rho)H_K$, $K \geq 2$. By substituting $S_2(\rho) = 1$, which is true by definition, in the above equation we have that $\beta(\rho) = (1 - \alpha(\rho))/H_2$, and thus,

$$S_K(\rho) = \alpha(\rho) + \frac{1 - \alpha(\rho)}{H_2} H_K, \quad K \geq 2 \quad (5)$$

where $\alpha(\rho)$ is to be determined.

Let us examine the behavior of $\alpha(\rho)$ as ρ approaches zero. In an almost empty system we know that $\lim_{\rho \rightarrow 0} T_K = H_K/\mu$, which from (4) suggests that $\lim_{\rho \rightarrow 0} S_K(\rho) = H_K/H_2$, $K \geq 2$. Consequently, by

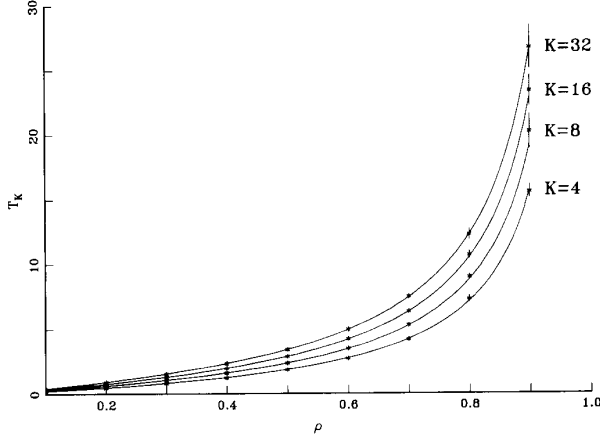


Fig. 2. Comparison of simulation and approximation.

taking the limit of (5), we get $\lim_{\rho \rightarrow 0} \alpha(\rho) = 0$. Using simulation results for $K = 4, 8, 16$, and 32 , we found that a good approximation to $\alpha(\rho)$ is given by $\alpha(\rho) \approx 4\rho/11$. Substituting this into (5) and (4) leads to the following approximate expression for T_K .

$$T_K \approx \left[\frac{H_K}{H_2} + \frac{4}{11} \left(1 - \frac{H_K}{H_2} \right) \rho \right] T_2, \quad K \geq 2. \quad (6)$$

Note that the scaling factor in the above expression exhibits a logarithmic behavior in K and a linear behavior in ρ . We used (6) and (1) to evaluate T_K for $K = 4, 8, 16$, and 32 . A comparison of approximate and simulation results is illustrated in Fig. 2. In this figure, we have set $\lambda = 1$ and have plotted 90 percent confidence intervals around the simulation points. The relative error in our approximation is less than 5 percent.

IV. APPLICATIONS

In this section, we apply our approximate analysis to two different computer system models. In the first model, we contrast two different multiprocessor configurations to determine the effect of synchronization delay in the fork/join queue. The second model leads to the extension of the basic fork/join queue that allows independent arrivals to all the processors of the system.

A. Two Multiprocessor Architectures

The fork/join configuration achieves parallelism on a task level, i.e., jobs are *split* into K statistically identical tasks that can be executed simultaneously on all K processors. The price that one pays for this increased parallelism is the synchronization delay that occurs when tasks wait for their siblings to finish execution. An alternate architecture for the K processors is to *not split* jobs but rather distribute all K tasks of a job randomly to one of the K processors. Since each job is assumed to consist of K independent exponentially distributed tasks, the queueing system formed by the nonsplitting architecture consists of a set of K independent $M/E_K/1$ queueing systems having a job arrival rate of λ/K jobs per unit time where each job consists of K exponential segments. It can be shown [7] that the mean response time for this system is given by

$$T_{M/E_K/1} = \left(K - \frac{K-1}{2} \rho \right) T_1.$$

The question naturally arises as to which architecture yields the minimum job mean response time. In Fig. 3, we plot the ratio of the mean response times for the no splitting case to that of the splitting case as calculated using the approximation derived in Section III, i.e., we plot the ratio $R_K(\rho) = T_{M/E_K/1}/T_K$. We see in this figure that the splitting case, for all values of K and ρ , has a lower mean response

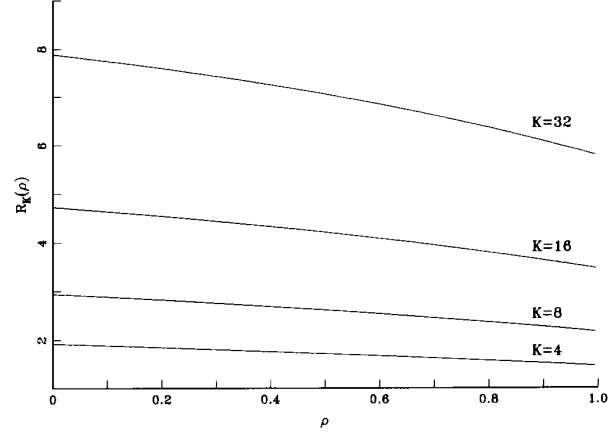


Fig. 3. Ratio of mean response times for no splitting to splitting.

time than that of the no splitting case and that the ratio is increasing with K . The intuition behind this result lies in the fact that there is a larger probability of having an idle processor when there is work at the other processors in the no splitting case than there is in the splitting case.

It is of interest to determine the limiting values of the ratio of $R_K(\rho)$ for low and high utilizations. For $\rho \rightarrow 0$ we can calculate this exactly since, for the fork/join queue, the exact delay is equal to the maximum of K exponentially distributed random variables whereas for the nonsplitting case it is equal to the sum of K exponential random variables. We thus have $\lim_{\rho \rightarrow 0} R_K(\rho) = K/H_K$. For $\rho \rightarrow 1$, substituting (1) into (6) and taking limits leads to $\lim_{\rho \rightarrow 1} R_K(\rho) = (6(K+1))/(6+7H_K)$.

B. Database Example

Here we consider an extension to the fork/join queue that models a distributed replicated database. We consider a system of K identical databases each servicing a queue of requests. Requests arrive to the system according to a Poisson process with rate λ . There are two types of requests, reads and writes, which occur with probability r and $w = 1 - r$, respectively. Read requests are randomly assigned to a database with a probability of $1/K$ whereas write requests are split into K identical requests, one being assigned to each of the database replications. A write operation is not considered to be completed until all the databases have been updated. Read requests, however, require no synchronization. The queueing model formed by this system is thus similar to that studied in Section III with the addition of an independent arrival stream to each of the K queues. Another model of a replicated database but with centralized control can be found in [8].

We analyze the replicated database model by extending the approximation of the previous section. We let $T_r(\lambda, \mu)$ and $T_w(\lambda, \mu)$ be the mean response time for read and write requests, respectively, and let $T_{\text{avg}}(\lambda, \mu) = rT_r(\lambda, \mu) + wT_w(\lambda, \mu)$ be the average response time for a request. We write T_K as $T_K(\lambda, \mu)$ in this section. It is clear, since the arrival process of requests to each of the databases is Poisson with a rate of $\lambda w + \lambda r/k$, that the mean response time for read requests is given by the solution to an $M/M/1$ queue with this arrival rate, namely

$$T_r(\lambda, \mu) = \frac{1}{\mu - \lambda(w + r/k)}.$$

The write requests must wait for synchronization and thus are similar to jobs modeled by the fork/join queue. To approximate $T_w(\lambda, \mu)$ we use equation $T_K(\lambda, \mu)$ with a modified service rate to account for the service capacity used by read requests. Specifically, we set

$$T_w(\lambda, \mu) \approx T_K(w\lambda, \mu - \lambda r/k).$$

The comparison of this approximation using (6) and (1) to simulation for $r = 0.75$ and $w = 0.25$ is shown in Fig. 4, where we have set $\lambda = 1$, and the approximation for the average response time, given by $T_{\text{avg}}(\lambda, \mu)$, is shown in Fig. 5. We see that $T_w(\lambda, \mu)$ is a very close match to the simulation points and that the mean write response time is increasing in K . This, of course, is a result of the increased synchronism delay in more updated copies of the database. It is interesting to note, however, that the mean response time does not vary significantly as a function of K . As K increases (decreases), it is clear that the write response time also increases (decreases) since more databases need to be updated. Read response time, however, decreases (increases) since the read arrival rate to each database is reduced (increased). These two conflicting components of the mean response time trade off against each other and cause the average to grow very slowly in K for Fig. 5.

V. CONCLUSION AND FUTURE RESEARCH

We have introduced an approximate analysis for the mean response time for a homogeneous fork/join queueing system consisting of K exponential servers. The approximation is notable for two main points: first, its development was guided by both experimental and theoretical considerations and second, it is represented as a simple, closed form, equation. The error of the approximation, as compared to simulation, was found to be less than 5 percent for systems where $K \leq 32$. Accurate simulation for larger K values was found to be computationally intractable. There is reason to believe that the approximation will also be good for higher values of K .

There is evidence that other queueing systems that are difficult to analyze exactly might be approximated using the method of this paper. We call this method the *scaling approximation*. We can summarize this method as follows: let P_K be a performance metric that we wish to calculate and suppose that $\lim_{K \rightarrow \infty} P_K \rightarrow \infty$. Furthermore, suppose that there exist upper and lower bounds that grow at a rate $O(f(K))$. Then it must be the case that P_K also grows at the rate $O(f(K))$ and we seek an approximation of the form $P_K \approx G(K)P_1$ where $G(K)$ is a function that grows at rate $O(f(K))$ and P_1 is an evaluation of the performance measure for $K = 1$ (if P_j is known, then one would approximate P_K as $P_K \approx G(K - j + 1)P_j$). We note that the values of P_1 could be obtained from other exact or approximate analysis or even by a careful simulation. The form of the function $G(K)$ is guided by the characteristics of the particular problem and the combination of boundary conditions and experimental simulation can determine parameter values of the function $G(K)$.

There are several extensions of this work that are of interest. Extending the basic model to the case where the servers are of different speeds would model a system consisting of nonhomogeneous processors. Furthermore, one could relax the exponential assumptions on the arrival and service processes and could allow for a fraction of the customers to feed back into the system, again thus modeling a job consisting of a series of fork/join cycles. In many cases it is not possible to break a job so that each processor receives a task. Allowing a distribution for the number of tasks that make up a job would allow one to analyze cases of this type. This in turn would raise questions about the scheduling of tasks and tradeoffs concerning the optimal splitting of jobs into tasks.

APPENDIX A

UPPER BOUND

In this Appendix, we derive the upper bound given in (2). Our derivation of the upper bound uses properties of *associated random variables* which we state without proof. The proofs for these properties can be found in [2].

Definition: Random variables $X = \{X_1, X_2, \dots, X_n\}$ are said to be *associated* if $\text{cov}[f(X), g(X)] \geq 0$ for all pairs of increasing functions f and g .

Properties:

P1) Increasing functions of associated random variables are associated.

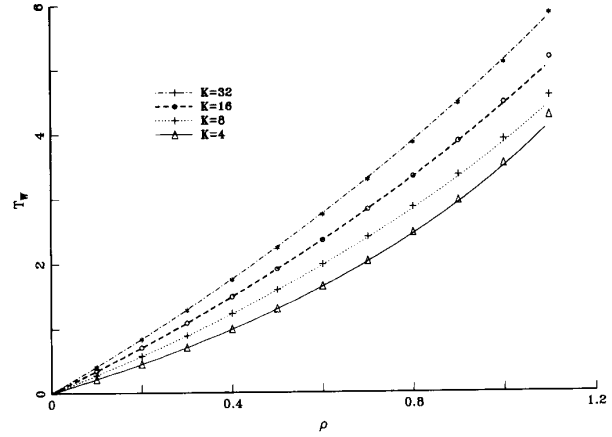


Fig. 4. Comparison of write response time from simulation and approximation.

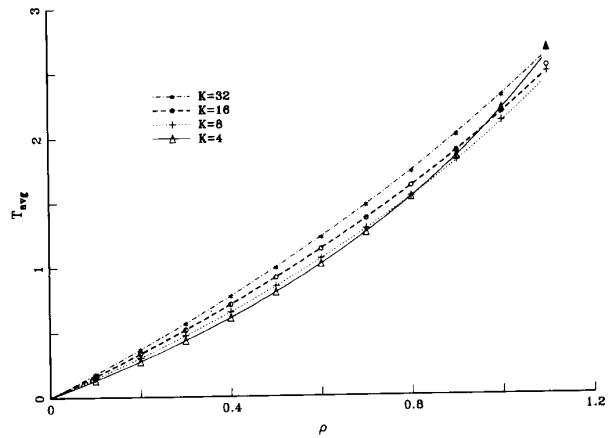


Fig. 5. Comparison of average response time from simulation and approximation.

P2) If two sets of associated random variables are independent of one another, then their union is a set of associated random variables.

P3) Independent random variables are associated.

P4) If $X = \{X_1, X_2, \dots, X_n\}$ are associated, then

$$P[\max_{1 \leq i \leq n} X_i > x] \leq 1 - \prod_{i=1}^n (P[X_i \leq x]). \quad (7)$$

Our derivation consists in showing that T_i^j , $j = 1, 2, \dots, K$, the response times for the i th customer through queue j , are associated random variables and then using (7) to calculate an upper bound.

Lemma 1: The random variables T_i^j , $j = 1, 2, \dots, K$, are associated.

Proof: We first establish that W_i^j , $j = 1, 2, \dots, K$, the waiting time of the i th customer through queue j , are associated. We prove this by induction.

Basis Step: The random variables W_1^j , $j = 1, 2, \dots, K$ are associated since, assuming that the system is initially empty, the equations $W_1^j = 0$, $j = 1, 2, \dots, K$, hold with probability 1.

Inductive Step: Assume that W_m^j , $j = 1, 2, \dots, K$, are associated for $m = 1, 2, \dots, i$. We will show that this implies that W_{i+1}^j , $j = 1, 2, \dots, K$ are associated. We have the following equality [7]

$$W_{i+1}^j = (W_i^j + U_i^j)^+, \quad j = 1, 2, \dots, K \quad (8)$$

where $U_i^j = x_i^j - t_{i+1}$, $j = 1, 2, \dots, K$, and $(x)^+ = \max(0, x)$. We identify three sets of associated random variables: by inductive hypothesis W_i^j , $j = 1, 2, \dots, K$ are associated, the set of random variables, U_i^j , $j = 1, 2, \dots, K$, are associated since they are independent random variables (P3), and the union of the sets W_i^j and U_i^j , $j = 1, 2, \dots, K$, are also associated since these sets are independent of one another (P2). Thus, W_{i+1}^j , $j = 1, 2, \dots, K$ are associated since (8) shows that they are given as increasing functions of associated random variables (P1).

The proof continues in a similar manner to the above by using the following equation.

$$T_i^j = W_i^j + x_i^j, \quad j = 1, 2, \dots, K. \quad (9)$$

Lemma 1 follows from (8) since x_i^j , $j = 1, 2, \dots, K$, are associated since they are independent (P3), the union of sets W_i^j and x_i^j , $j = 1, 2, \dots, K$, are associated (P2), and, from (8), T_i^j , $j = 1, 2, \dots, K$, is an increasing function of associated random variables and thus is associated (P1). ■

We can now derive the upper bound for our system.

Theorem 1: Let $T_K \equiv E[\lim_{i \rightarrow \infty} \max_{1 \leq j \leq K} T_i^j]$ be the expected response time of the queueing system defined in Section II. Then the following upper bound holds.

$$T_K \leq H_K \frac{1}{\mu - \lambda}. \quad (10)$$

Proof: From Lemma 1 we know that T_i^j , $j = 1, 2, \dots, K$, are associated. Using (7), and taking limits, we thus have

$$G(t) \equiv \lim_{i \rightarrow \infty} P[\max_{1 \leq j \leq K} T_i^j > t] \leq 1 - \prod_{j=1}^K \lim_{i \rightarrow \infty} P[T_i^j \leq t]. \quad (11)$$

Since queues are identical, we can define $F(t) \equiv \lim_{i \rightarrow \infty} P[T_i^j \leq t]$. Using this in (11) and integrating yields

$$T_K = \int_0^\infty G(t) dt \leq \int_0^\infty (1 - F^K(t)) dt. \quad (12)$$

The theorem then follows from the fact that for our system $F(t) = 1 - e^{-(\mu - \lambda)t}$ and that $\int_0^\infty (1 - (1 - e^{-t})^K) dt = H_K$. ■

APPENDIX B

TWO QUEUES

In this Appendix, we consider the system described in Section II with $K = 2$ queues and obtain the mean job response time. The response time of a task consists of two components: 1) the time spent in an M/M/1 queueing system and 2) the time spent in the system after service completion and before leaving the system. The latter is called synchronization delay; it is nonzero for the task that finishes its service first and zero for its sibling. Since each job consists of exactly two tasks, the mean job response time equals the mean task response time. Thus, the mean job response time in our two-queue system is the sum of the mean response time in an M/M/1 queueing system, $T_1 = 1/(\mu - \lambda)$, and the mean synchronization delay S , i.e.,

$$T_2 = T_1 + S. \quad (13)$$

We now proceed to evaluate the mean synchronization delay S . Using Little's formula, we may write $S = N/2\lambda$, where N is the average number of tasks which have completed and are waiting for their siblings to complete. Let k be any integer and define q_k to be the probability that the number of tasks in the first queueing system exceeds the number of tasks in the second queueing system by k . By symmetry, we have $q_k = q_{-k}$, $k \geq 1$ and thus $N = 2 \sum_{k=1}^\infty k q_k$ which implies

$$S = \frac{1}{\lambda} \sum_{k=1}^\infty k q_k. \quad (14)$$

By writing the balance equation for the system, one can show that $q_k = q_{k+1} + p_{k,0}$, $k = 0, 1, \dots$, where $p_{k,0}$ is the probability that there are k tasks in the first queue and 0 tasks in the second queue. This implies that

$$q_k = \sum_{i=k}^\infty p_{i,0} \quad k = 0, 1, \dots. \quad (15)$$

Substituting (15) into (14) and simplifying yields

$$S = \frac{1}{\lambda} \sum_{i=1}^\infty \frac{i(i+1)}{2} p_{i,0}. \quad (16)$$

The generating function for $p_{i,0}$ has been obtained in [3] and is given by $P(z, 0) = (1 - \rho)^{3/2} / \sqrt{1 - \rho z}$. Using this to determine the first two moments of $p_{i,0}$ and substituting into (16) and (13) yields

$$T_2 = \frac{12 - \rho}{8} T_1. \quad (17)$$

It is interesting to note that the ratio $T_2/T_1 = (12 - \rho)/8$ varies over a very small range, i.e., from $11/8 = 1.375$ at $\rho = 1$ to $3/2 = 1.5$ at $\rho = 0$.

ACKNOWLEDGMENT

We are thankful to P. Heidelberger and D. Towsley for suggesting the use of associated random variables in proving the upper bound.

REFERENCES

- [1] F. Baccelli and A. M. Makowski, "Simple computable bounds for the fork-join queue," in *Proc. John Hopkins Conf. Inf. Sci.*, John Hopkins Univ., 1985.
- [2] R. E. Barlow and R. Proschan, *Statistical Theory of Reliability and Life Testing*. Holt, Reinhart and Winston, 1975.
- [3] L. Flatto and S. Hahn, "Two parallel queues created by arrivals with two demands I," *SIAM J. Appl. Math.*, vol. 44, pp. 1041-1053, Oct. 1984.
- [4] L. Flatto, "Two parallel queues created by arrivals with two demands II," *SIAM J. Appl. Math.*, vol. 45, pp. 861-878, Oct. 1985.
- [5] P. Heidelberger and K. S. Trivedi, "Queueing network models for parallel processing with asynchronous tasks," *IEEE Trans. Comput.*, vol. C-31, pp. 1099-1109, Nov. 1982.
- [6] —, "Analytic queueing models for programs with internal concurrency," *IEEE Trans. Comput.*, vol. C-32, pp. 73-82, Jan. 1983.
- [7] L. Kleinrock, *Queueing Systems, Vol. 1: Theory*. New York: Wiley, 1975.
- [8] R. D. Nelson and B. R. Iyer, "Analysis of a replicated data base," *Perform. Eval.*, vol. 5, pp. 133-148, Aug. 1985.

A Simple Method for Determining Hadamard Sequency Vectors

MANSOUR I. IRSHID

Abstract—A simple method for determining the sequency ordering of any row in any Hadamard matrix directly from its binary representation is developed. This proposed method is proved to be much simpler than the well-known bit-reverse inverse Gray code method.

Manuscript received September 12, 1985; revised November 14, 1986.
The author is with the Department of Electrical Engineering, Yarmouk University, Irbid, Jordan.
IEEE Log Number 8716231.